

1교시: 로컬 정적 서버 실행 - browser와 curl 확인

수업 목표

- 정적 파일 서버를 실행하고 HTTP로 확인한다.
- process, port, file path, HTTP status를 하나의 evidence로 연결한다.
- Day3에서는 미니앱 구현을 시작하지 않고, 최소 정적 파일만 사용한다.

50분 흐름

Time	Activity
0-5분	Day2 repository와 CLI evidence 확인
5-15분	정적 서버와 동적 앱의 차이 설명
15-30분	index.html 생성, 서버 실행, browser/curl 확인
30-40분	server terminal log와 HTTP status 기록
40-50분	port 충돌/경로 오류와 다음 교시 연결

0-5분 Day2 repository와 CLI evidence 확인

- 진행: Day2 repository와 CLI evidence 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

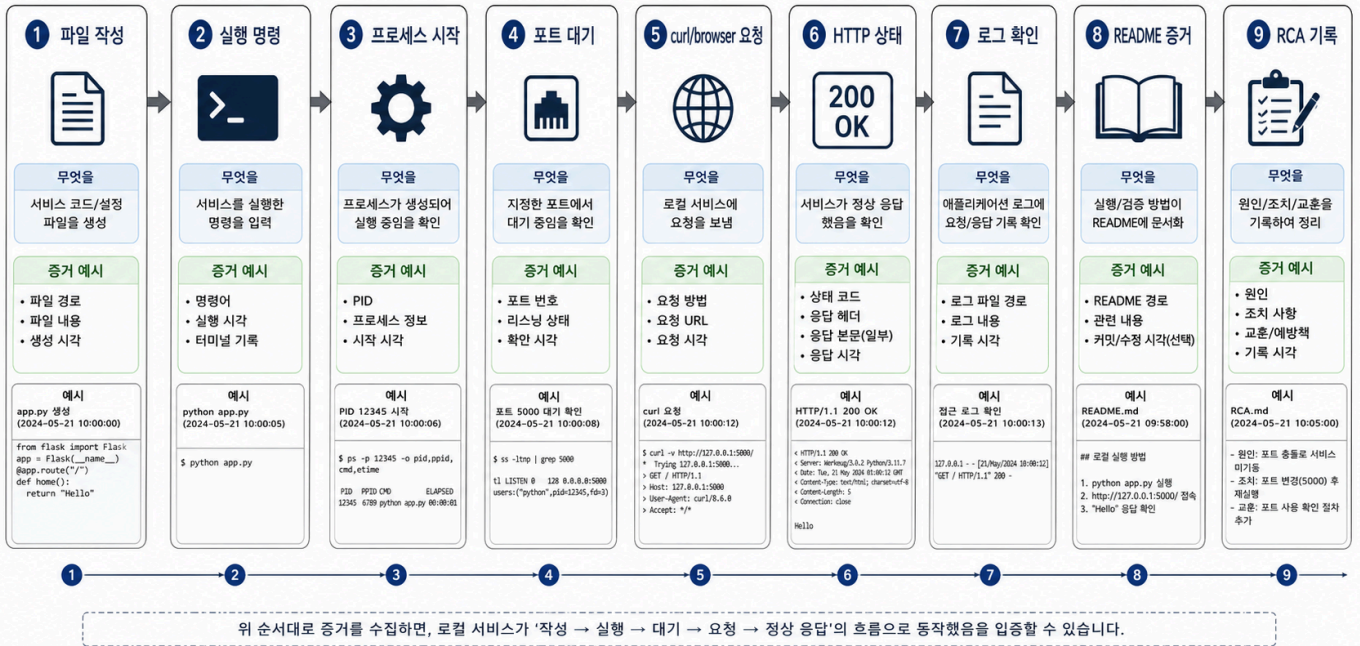
상세 설명

정적 서버는 HTML, CSS, image 같은 파일을 그대로 HTTP로 제공한다. 오늘 만드는 `index.html` 은 앱 구현이 아니라 서버가 file path를 읽고 HTTP response를 반환하는지 확인하기 위한 최소 자료다. 로그인, API, 데이터베이스, JavaScript 기능은 시작하지 않는다.

`python3 -m http.server 8000` 은 현재 directory를 기준으로 파일을 제공한다. 그래서 서버를 어느 path에서 실행했는지가 중요하다. 같은 명령이라도 repository root에서 실행하면 root의 파일을 제공하고, 다른 directory에서 실행하면 다른 파일 목록을 제공한다.

Visual 1: Service evidence flow

로컬 서비스 실행 증거 흐름



그림을 볼 때는 "명령을 실행했다"에서 멈추지 말고 path -> process -> port -> HTTP status -> server log 순서로 evidence가 이어지는지 확인한다. 빈칸이 있으면 다음 사람이 같은 결과를 재현하기 어렵다.

5-15분 정적 서버와 동적 앱의 차이 설명

- 진행: 정적 서버와 동적 앱의 차이 설명
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

시작 상태

Day2 repository 안에서 다음 파일만 만든다.

```
<!doctype html>
<title>Week 1 Lab</title>
<h1>Week 1 Local Service</h1>
```

Visual 2: Browser와 curl이 보는 것

확인 도구	보는 위치	남길 evidence
browser	화면에 렌더링된 HTML	URL, 화면에 보인 제목
curl -I	HTTP header와 status	200, 404 같은 status
server terminal	process가 받은 request	GET / log excerpt
pwd	서버 command 실행 위치	repository 기준 상대 path

브라우저 화면은 사용자 관점 evidence이고, curl 과 server log는 운영 관점 evidence다. 둘 중 하나만 기록하지 말고 같은 요청을 서로 다른 각도에서 확인한다.

15-30분 index.html 생성, 서버 실행, browser/curl 확인

- 진행: `index.html` 생성, 서버 실행, `browser/curl` 확인
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

명령 절차

```
pwd
printf '<!doctype html>\n<title>Week 1 Lab</title>\n<h1>Week 1 Local Service</h1>\n' > index.html
python3 -m http.server 8000
```

서버를 실행한 terminal은 그대로 둔다. 새 terminal에서 확인한다.

```
curl -I http://localhost:8000
curl http://localhost:8000
```

서버 종료는 실행 중인 terminal에서 `Ctrl+C` 를 사용한다.

확인 질문

- 서버를 실행한 directory는 어디인가?
- `browser`와 `curl -I` 은 각각 어떤 evidence를 제공하는가?
- `Ctrl+C` 후 다시 `curl` 하면 어떤 증상이 나와야 하는가?

30-40분 server terminal log와 HTTP status 기록

- 진행: server terminal log와 HTTP status 기록
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

다음 주차 매핑

Docker에서는 이 실행이 container command가 되고, Kubernetes에서는 Pod 안 process가 되며, AWS에서는 hosting 대상 artifact가 된다. Terraform은 나중에 port와 hosting 리소스를 선언한다.

예상 결과

- `curl -I http://localhost:8000` 은 200 status를 보여야 한다.
- `curl http://localhost:8000` 은 Week 1 Local Service 가 포함된 HTML을 출력해야 한다.
- 서버 terminal에는 GET / 요청 log가 찍힌다.
- port 8000이 이미 사용 중이면 address already in use류 오류가 난다.

흔한 오해

오해	교정
<code>index.html</code> 을 만들었으니 앱 구현을 시작한 것이다.	오늘 파일은 정적 서버 확인용 최소 파일이다. 기능 개발은 Day4 이후다.
브라우저가 열리면 모든 evidence가 충분하다.	URL, status, server log, 실행 path를 함께 기록한다.
8000번 port는 항상 비어 있다.	다른 process가 사용 중일 수 있다. port 충돌은 흔한 운영 증상이다.

40-50분 port 충돌/경로 오류와 다음 교시 연결

- 진행: port 충돌/경로 오류와 다음 교시 연결
- 완료 조건: 아래 자료를 사용해 이 시간 블록의 산출물을 만든다.

실습 Evidence

Evidence	Value
command	python3 -m http.server 8000
path	
port	8000
HTTP status	
server log excerpt	

학술 근거와 현업 DevOps insight

분산 시스템을 배우기 전에는 가장 작은 HTTP service를 정확히 관찰할 수 있어야 한다. SRE 관점에서 symptom은 사용자가 보는 결과이고, cause는 process, port, file, config 중 하나일 수 있다. 정적 서버 실습은 이 구분을 낮은 비용으로 훈련한다.

평가 기준

기준	2점 evidence
50분 참여	시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.
증거 산출	수업에서 요구한 note, command, table, blocker 중 해당 산출물을 구체적으로 남겼다.
전이 연결	오늘 개념이 Week2~6 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다.

공식/학술 근거 링크

- MDN HTTP Overview, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> - local service 확인을 HTTP request/response evidence로 연결한다.
- RFC 9110: HTTP Semantics, <https://datatracker.ietf.org/doc/html/rfc9110> - status code와 resource 확인의 공식 기준이다.
- MIT Missing Semester, <https://missing.csail.mit.edu/> - shell command와 debugging 기록을 실무형 computing literacy로 다루는 근거다.